

Livestream App — Trust, Privacy & Communication

A trust review of a privacy-first live-streaming app: video is encrypted in the browser, discovery uses no central server, and no accounts or IP logs are kept. This document traces who can see what, on every hop — written to expose residual leaks, not to reassure.

Choose your path

The app isn't a single privacy setting — it's a set of paths, from "open a link" to "trust no shared infrastructure." Pick the row that matches your threat; the rest of this document is the evidence behind each one. **Every party in every row is content-blind** — none can decode your video — so the columns below are about *who you still rely on to deliver the code and coordinate the stream*, and what each step removes.

Path	Choose it if you want...	Who you still rely on (all content-blind)	Effort
1. Default — the two hosted pages	Private video + no accounts, zero setup	The code host (hash-pinned), the broker, the relay fleet — none can decode; they see your IP only while you're connected	Open a link
2. + Tor / VPN	Also hide your IP / location from every hop	The above, minus IP exposure (you trust your VPN/Tor instead)	Run a VPN
3. Own your copy — self-contained download	Control your own app delivery	The broker + relay fleet only — no code host ; the code is yours, checkable against the published hash	Download + run/host
4. + your own access key	Your own usage tier and your own revocation switch	Same as 3	Request / mint a key
5. Owned + your own relay  roadmap	No shared dependency at all	Only a relay you chose or run — nothing of ours	Roadmap (config switch pending)

These stack: add **Tor/VPN** to any row, **self-host the discovery relay** on any owned path, and **rotate keys / mint a fresh identity** at any time. The full lever-by-lever detail — what each closes, how, and its honest limit — is §10.

What every path guarantees

- **Your video is encrypted in your browser.** No relay or broker can decode it — the key lives only in the `#...` of your share link and never reaches a server.
- **No accounts, no chat, no directory to browse.** Discovery is a signed record on a public distributed hash table (DHT) — a stream is reachable only to someone who already has its key.
- **No IP is stored.** Every hop sees your IP *while you're connected* (unavoidable in software); none of ours logs or rolls it up.
- **The client is open source** — and in paths 3–5 it's hosted by you.

Key terms (plain language)

- **End-to-end encrypted** — scrambled on the sender's device and unscrambled only on the recipient's; nothing in between holds the key.
- **DHT (distributed hash table)** — a decentralized, serverless key→value lookup (the same kind of network BitTorrent uses to find peers). Here it maps a stream's public key to its relay, so no server holds a list of streams.
- **Content key (`#...`)** — the secret that decrypts the video. It lives only *after the #* in the share link, and browsers never send that part of a URL to any server.
- **Trusted computing base (TCB)** — the one component that, if it were malicious, would defeat every other protection. For a web app, that's whoever serves the client code.
- **Subresource Integrity (SRI)** — the page pins the exact cryptographic hash of the code it loads; the browser refuses any file that doesn't match, so the code host can't silently swap it.

For reviewers: this document is written to be *checked*, not to reassure. §2 is the trust boundaries, §6 the code-verified who-knows-what, §7 the ranked residual leaks, §8 the hard floors we can't engineer away. Tell us where the model is wrong, or where a path doesn't actually deliver the goal it claims — that feedback is the point.

HOW IT WORKS — ONE BROADCAST, END TO END

Broadcaster's browser

encrypts every video frame before it leaves the device

↓ encrypted video

Origin relay

moves bytes it cannot read

↓ encrypted video (relay to relay)

Edge relay

nearest the viewer — also cannot read

↓ encrypted video

Viewer's browser

decrypts locally with the key from the share link

Broker

assigns relays and issues short-lived access tokens. Cannot decrypt.

Discovery (DHT)

maps a stream's public key → its relay. No server holds a list.

The key (#...)

lives only in the share link — browser to browser, never a server.

① The broadcaster's browser mints a keypair and a content key, asks the broker for a relay, and signs **public key → relay** into the DHT. ② A viewer opens the share link, looks the public key up on the DHT, and gets a nearby relay plus a short-lived token from the broker. ③ Video flows relay-to-relay as ciphertext. ④ Only the two browsers hold the key from the link, so only they can decode.

1. Goal, stated as a threat model

Two guarantees for end users (broadcasters **and** viewers):

- **Content confidentiality** — no party in the middle (a relay, the broker, or *whoever serves the client code*) can decode the media. Only the two browsers can.
- **Identity protection** — no party can trace a stream to a person, *even if a powerful entity compels or seizes* the code host, the broker, or a relay.

Out of scope (accepted limits): copy protection after decode (an authorized viewer can re-capture decoded frames — this is not DRM); and network-level anonymity of a browser's IP from any server it connects to directly (that is the user's own tool — Tor/VPN).

You can only be compelled to reveal what you retain and can enumerate. The two levers are *data minimization* (don't store it) and *compartmentalization* (split knowledge so no single party links identity ↔ broadcast ↔ viewer). Every design choice below serves one of these.

2. Parties and trust boundaries

- **Browser endpoints** (*trusted*) — the broadcaster and the viewer. They mint and hold the content key (the `#...` in the link), the cryptographic identity, and the rotate secret; they see plaintext; they sign the discovery record; they encrypt/decrypt every frame. They also carry a **public access key** (see §5) — not a secret.
- **Page host — anywhere** (*varies*) — whoever serves the two pages (`send.html` / `view.html`): the app's own site, a third party's web server, or nobody at all (a page opened as a local file). It serves only the two HTML shells plus the public access key; it holds **no secret**, does **no coordination**, and sees only the initial page load (nothing at all for a local-file open).
- **Code host** (*the client's TCB*) — serves the JavaScript bundle the pages load. It cannot decode, but whoever serves the code can in principle backdoor it, so it is the browser's **trusted computing base**. The pages pin the exact bundle hash (**Subresource Integrity**), so the code can't be silently swapped. **In owned mode this party disappears entirely**: the self-contained bundle carries all its own code — nothing is fetched from us — so there is no code host to trust; the code is whatever you downloaded, and you can check it against the published hash. See §10.
- **Broker** (*central, ours*) — a small coordination service (a serverless worker plus a database and a key-value store). The browser calls it directly to reserve a relay and fetch salts. It **issues** the per-broadcast access token and serves the salts. It is content-blind (never sees the `#...` key). Because the browser calls it directly, it **sees the browser's IP** while the request is in flight.
- **Relay** (*untrusted*) — a minimal unikernel with no persistent disk and no filesystem to log to. It moves ciphertext and sees a peer's IP:port transiently, as live network state. No disk, no access log, no key.
- **Discovery relays** (*third-party*) — public gateways that bridge ordinary HTTP to the DHT. A gateway sees the resolving browser's IP alongside the public key it looks up. It **cannot forge** records — they are cryptographically signed.

Media path: broadcaster browser → origin relay (ciphertext) → edge relay (relay-to-relay pull) → viewer browser. **Discovery:** the broadcaster signs a record to the DHT (public key → origin relay); the viewer reads it off the DHT, then calls the broker to place an edge. **The content key never enters any of these hops** — it travels only in the share link's `#...` fragment, browser to browser.

3. Content confidentiality — the key never touches a server

Media is encrypted with **AES-256-GCM** per encoded chunk (`src/crypto/media-crypto.ts`), for **both audio and video** (two build-time seams; the build fails closed if either is missing). A relay forwards only `[timestamp][nonce][ciphertext+tag]` . The codec/resolution catalog is left in the clear by design — it leaks stream *metadata* (resolution, codec), never content.

The key is **derived in the browser** with a standard key-derivation function:

```
contentKey = HKDF-SHA256( input = linkKey, salt = globalSalt || streamSalt, info =  
"media-v1|<publicKey>|<epoch>" )
```

- **linkKey** — 32 random bytes minted by the broadcaster's browser, carried **only** in the `#...` fragment of the share link (`.../view.html?node=<publicKey>#k=<key>`). Browsers never transmit the fragment of a URL, so it reaches no server — relay, proxy, or access log — by the URL spec. Its real exposure surface is the *link as a string* (the channel you share it over, browser history/sync, extensions, URL shorteners), not the streaming path — see §7.
- **streamSalt** — per-broadcast, broadcaster-rotatable (the *reset key* button). Held in the broker's key-value store with an expiry, owner-gated by a rotate secret kept only in the broadcaster's browser.
- **globalSalt** — the operator's kill switch (rotated from the admin console). Rotating it re-keys every live stream at once.

The salts are not secret. They decode nothing without the `#...` fragment, so the broker serving them does not weaken content-blindness — no party but the two browsers can derive the content key. Rotating a salt is a **revocation seam**: it re-keys future frames. (Today salts are served openly, so a cut-off viewer can re-fetch them; it becomes a true kill switch once salt delivery is access-gated — see §7.)

4. Discovery — a signed DHT record, not a server list

The broadcaster's **public key** is the stream's identity, its relay track name, and its DHT key. It is minted and kept in the broadcaster's browser storage (a stable link and stream ownership across reloads); a "new identity" action rotates to a fresh, unlinkable key. The private key never leaves the tab.

- **Publish:** the broadcaster signs a record and publishes it to public DHT gateways (pkarr — *public-key addressable records*): `publicKey → { origin relay address, access = public }`. The record is signed, so it cannot be forged.
- **Resolve:** a viewer reads that record straight off the DHT and verifies the signature. **No directory server; discovery is key-gated** — a stream is reachable only to someone who already holds its public key. The broker keeps no record of which streams ran (see §6).

Reserving a relay and getting a token happens **directly against the broker**: the browser posts to `/cdn/assign`, and the broker reserves a relay, reads the origin relay's address, and issues a scoped token — see §5.

5. Communication flows (wire-level, with privacy annotations)

Every request the browser makes to the broker carries a **public access key** (`Bearer pk_...`). This key is public *by design*: it ships in the pages, identifies a *usage tier* (not a user), and its safety is capability + limits, not secrecy — it authorizes only rate-limited, self-serve relay reservation and token issuance for one tier's broadcasts. The operator can **rotate** it to invalidate every copy instantly.

Broadcaster go-live

1. **store salt** — `PUT /pub/salt/<publicKey> {stream salt, rotate secret}`. *Broker sees the salt (benign — decodes nothing) and the broadcaster's IP (§7).*
2. **derive content key** — in-browser, from the `#...` key + salts. *Never leaves the browser.*
3. **reserve a relay** — `POST /cdn/assign {broadcast = publicKey, role: publish} + access key`. The broker reserves an origin relay, reads its address, and returns `{ relay, origin address, token }`. *Broker sees the broadcaster's IP + coarse geo from the incoming request. It issues the publish token itself.*
4. **connect** — to the origin relay over WebTransport (QUIC in the browser). *Relay sees the broadcaster's IP (network floor). Media is ciphertext; the token's scope is this public key only.*
5. **announce** — sign and publish the DHT record (public key → origin relay). *Discovery gateways see the broadcaster's IP alongside the public key; the record is world-readable and durable (no content).*

Viewer watch

1. **read the key** — from the share link (required; no key → nothing to watch). *Local only.*
2. **resolve** — look the public key up on the DHT. ⚠️ *A discovery gateway sees the viewer's IP alongside the public key — a viewer↔content linkage to third-party infrastructure. Mitigation: Tor/VPN, or self-host the gateway (§7).*
3. **reserve a relay** — `POST /cdn/assign {broadcast = publicKey, role: watch, origin address} + access key`. The broker issues the subscribe token and places the viewer, returning `{ relay, token }`. **Placement:** if the viewer's nearest relay is the origin's relay, the broker co-locates them (direct connection, no extra hop); otherwise it stands up an edge relay that pulls from the origin. *Broker sees the viewer's IP + coarse geo (§7).*
4. **fetch salt** — `GET /pub/salt/<publicKey>`; derive the content key. *Broker sees salt (benign) + IP.*
5. **connect** — to the edge relay, subscribe under the public key. *Relay sees the viewer's IP (network floor). Ciphertext only.*

Token, key & abuse shapes

- **Tokens are clean:** `{ publish: [<publicKey>] | [], subscribe: [<publicKey>], expiry }` — no IP, geo, email, or account id; only the pseudonymous public key and an expiry. Issued by the broker and validated at the relay against a pushed public verify-key.
- **The access key is public;** abuse is bounded server-side: a per-key + per-IP rate window (default 60/min, override per tier) plus an optional CAPTCHA on publish. Revoke = rotate the key.
- **Geo** comes from the incoming request's own IP at the edge — no forwarding, no stored coordinates.

6. What each party knows (code-verified)

6a. Relay — *untrusted*

The relay never reads, logs, or reports the client IP; the autoscaler counts connections only. It is a unikernel with no persistent disk. **Sees (transient):** peer IP:port as live network state; the token's subscribe scope (= the public key); byte counts. **Ceiling:** a root operator could packet- capture "IP X is watching public key Y, right now" — unavoidable in any software; the user's mitigations are Tor/VPN plus an unlinkable public key.

6b. Broker — *central, ours*

The browser calls it directly, so it **sees the broadcaster's and viewer's IP** (from the incoming request). It receives the public access key (a usage tier, not a user), the broadcast's public key, and — for a viewer — the origin relay address. It **issues** per-broadcast transport tokens (and cannot decode — it never sees the `#...` key). It persists only **non-identifying operational samples** for capacity and billing (relay-seconds and connection/egress counts per box and per tier) — **no broadcast public key, no per-viewer data, no stored IP**. The abuse-limit counter is keyed by a **one-way hash of the IP** (HMAC under a server secret — the raw address is never stored) and expires in ≈ 2 min. *(An earlier build kept a per-broadcast lifecycle log and a per-broadcast usage row; both were identity-bearing and analytics-only, and have been removed — the broker no longer records which streams were live.)*

6c. Page host — *varies*

Serves only the two static HTML shells plus the public access key. **Holds no secret, does no coordination, serves no salts, signs no tokens, sees no media connection**. For a local-file open it isn't involved at all. The residual power is the *code host* (§8). In *owned* mode the page host also serves the code — but its **published hash** lets anyone confirm the bytes are the published build, so a host that tampers is detectable, not trusted.

6d. Data-at-rest inventory (what a seizure yields)

Store	Holds	Identity-bearing?
broker salts	global + per-stream salts (+ an owner hash), expiring	No — salts decode nothing without the #... key
broker rate counters	per-(key, hashed-IP) request counts, ≈ 2 min expiry	No — one-way IP hash, auto-expires
broker operational samples	relay-seconds + conn/egress counts, per box + tier	No — no broadcast key, no IP, no viewer
broker billing samples	tier + relay-seconds	Tier-level only
broker signing key	the token-signing private key	Issues transport tokens; cannot decode
DHT record (public)	public key $\rightarrow$ origin relay address; world-readable, durable	Pseudonymous; no content, no person
broker key-request list	a prospective operator's name + email + note	Ordinary contact info (opt-in form only; never a viewer)
relay	<i>nothing persisted</i>	—
page host	the two static files + public access key	None
code host	the client bundle (+ audio-codec wasm)	None (public code)

7. Residual exposures (ranked) + recommendations

- **MEDIUM/HIGH — the broker sees the browser IP.** Calling the broker directly is what makes geo accurate and removes an extra hop, but it hands the broker the broadcaster's and viewer's IP (transiently; not rolled up; the abuse counter stores only a one-way IP hash and auto-expires in ≈ 2 min). *Mitigations:* Tor/VPN; or offer an opt-in same-origin relay hop for callers who want the app to shield their IP (trades the two-file simplicity).
- **MEDIUM — publish-collision under a public key.** Because the access key is public and the broker will issue a *publish* token for any broadcast name, a griever could publish-collide on someone else's public key. They **cannot inject valid media** (no #... key; content is end-to-end) — only disrupt availability. *Planned fix:* self-signed publish tokens (the relay verifies

against the broadcast's own public key), which makes publish authority cryptographic and removes the broker's signing key entirely; viewers then need no token at all.

- **MEDIUM — discovery IP ↔ public key.** A third-party gateway sees the resolving browser's IP and the public key. Records are signed (unforgeable) and live on the public DHT (the gateway is a swappable bridge). *Mitigations:* Tor/VPN; or self-host the gateway (a one-line change in `src/crypto/dht.ts`).
 - **MEDIUM — browser IP at the relay (network floor).** Any relay terminating a connection inherently sees the peer IP. Not removable in software; the user's tool is Tor/VPN.
 - **MEDIUM — the code host is the client's TCB.** In the default (thin) mode, one server delivers the JavaScript. Mitigated (not eliminated) by **enforced Subresource Integrity** — the pages pin the exact hash and the host serves the code with the headers that make the browser actually validate it, so the code can't be silently swapped — plus the code is small, open-source, and inspectable. **Eliminated** in *owned* mode (§8, §10): the self-contained copy loads no code from us.
 - **MEDIUM — the share link's distribution channel.** The `#...` key never appears in an HTTP request, so it's absent from relay, proxy, and access logs (that's the whole reason it lives in the fragment). But the *link itself* is only as private as how you share it: whatever channel carries it (messenger, email), your browser history/sync, bookmarks, extensions, and any URL shortener you run it through all see the full link — and a backdoored client could read `location.hash` (the code-host TCB again, §8 floor 2). *Mitigations:* share over a private channel, avoid shorteners, use owned mode + a browser you trust.
 - **LOW — key-request contact info.** The "request a key" form stores a prospective *operator's* name
 - email (+ note) — ordinary contact info, never tied to a viewer or a broadcast. Only exists if someone asks for their own key.
 - **LOW — precise geo at the broker.** The broker sees the browser's real coarse geo. *Recommend:* coarsen to state / country before it reaches the ranker.
 - **LOW — catalog metadata.** The codec/resolution catalog is cleartext by design (never content).
 - **Benign — the broker sees salts.** By construction salts ≠ the `#...` key, so it cannot decode.
-

8. The hard floors (be honest with users)

1. **IPs are visible at the edges.** A relay, a discovery gateway, and the broker each see the peer IP; you can decline to *store* it (we do), not to see it. Network anonymity is the user's tool (Tor/VPN).
 2. **Someone delivers the client code.** On the plain web, whoever serves the JavaScript can backdoor it per-user, undetectably — that server is the trusted computing base. The app narrows this three ways: the pages can be **hosted anywhere or opened locally**; they **pin the exact code hash** (Subresource Integrity), so the default host can't swap it; and the whole client is open-source and small. **Owned mode removes it:** you run your own downloaded copy, so *no* code loads from us at all, and you can check that copy against the **published hash**. The remaining gap to a *fully verifiable* client is a **reproducible build** (proving those bytes correspond to the published source) — the one open item on this floor.
 3. **Prospective surveillance.** Minimization defeats "hand over your history," not "start logging now" for a party that still runs live infrastructure the target uses. The broker is such a party and sees IPs while connected — so the structural mitigations (Tor/VPN, self-host the discovery gateway, self-signed tokens, bring your own relay) are what raise the floor.
 4. **Monetization reintroduces identity.** Paid accounts create person ↔ public key ↔ payment records. If anonymity matters, decouple payment from broadcast identity (prepaid / blind tokens), or accept that a paid tier is not an anonymous tier.
-

9. Status at a glance

Property	Today	How to strengthen
A relay can decode content	No ✓	—
The broker can decode content	No ✓	—
The code host can decode content	No ✓	—
A server holds a list of broadcasts	No — DHT discovery ✓	—
The app holds any secret / coordinates	No — two static files ✓	—
A central token-signing key exists	Yes ⚠	Roadmap: self-signed tokens remove it
Publish-collision resistant	No — public key can be reserved ⚠	Roadmap: verify publish against the broadcast's own key
The broker sees the browser IP	Yes — direct call ⚠	Tor/VPN, or an opt-in shielding hop
A viewer IP is stored anywhere	No — only a ≈ 2 min one-way-hashed counter ✓	—
Precise geo is stored	No ✓	—
Precise geo seen by the broker	Yes ⚠	Coarsen to state / country
Real identities stored	No — no accounts ✓	—
Chat metadata	None — no chat ✓	—
Key revocation lever	Per-stream + global salt; rotate the access key	True kill switch once salt delivery is access-gated
Discovery gateway sees IP $\leftrightarrow$ public key	Yes ⚠	Self-host the gateway
Who can backdoor the client (TCB)	Thin mode: the code host (hash-pinned) ⚠ · Owned mode: nobody (you host it) ✓	Reproducible build closes the last gap

Property	Today	How to strengthen
The client is verifiable	Partial — hash-pinned · owned copy · open source	Full — reproducible build vs the published hash

10. Privacy options you control (a menu of levers)

Everything above describes the *defaults*. Beyond them, a user or operator can pull specific levers to harden a specific exposure. Each is independent — stack the ones that match your threat. Ranked roughly from "anyone, in seconds" to "most independence."

Lever	Closes / hardens	How	Cost / honest limit
Tor / VPN	The IP every hop sees (broker, relays, discovery) — §8 floor 1	Run the browser behind Tor or a VPN; encryption + discovery are unchanged	Latency; you trust your VPN/Tor instead
Fresh identity	Linking two of your streams to one broadcaster	The "new identity" action mints a new, unlinkable public key (new share link)	Old links stop working (that's the point)
Rotate / kill switches	A leaked link, or a stream you want cut off	In-app <i>reset key</i> (per-stream salt) · operator <i>global salt</i> (re-keys all live streams) · rotate the access key (kills every copy)	Salt delivery isn't access-gated yet, so a cut-off viewer can re-fetch salts (§7) — a true kill switch lands with gating
Bring your own access key	Sharing one tier's quota / rate-limit / revocation with strangers	Request a key, or an operator mints one (add a tenant + enable self-serve)	Still rides our broker + relay fleet; the key is a <i>tier tag</i> , not anonymity
Own your copy	The code-host TCB — a compromised or compelled host backdooring the client (§2, §8 floor 2)	Download the self-contained bundle and run it off your own computer (double-click <code>send.html</code>), or host the folder on your own site — no code loads from us; check it against the published hash	You host it; the media path still uses our broker + relay fleet. A reproducible build (to prove the download matches the source) is roadmap
Self-host the discovery gateway	Discovery IP ↔ public key leaking to third-party gateways (§7)	Point at your own gateway (one line in <code>src/crypto/dht.ts</code>); records are DHT-native + signed, so the gateway is just a swappable bridge	You run a gateway; other hops unchanged
Bring your own relay  <i>roadmap</i>	The last shared dependency — the broker's relay reservation + DHT coordination	A config switch (not yet built) points an owned copy straight at an open Media-over-QUIC relay — e.g. Cloudflare's, a hosted one such as moq.pro (free up to 500 GB), or one you run yourself — with no broker and no access key; discovery via the share link	The client already supports open relays; the switch + putting the salt in the link are the remaining work. Then <i>no part of the path is ours</i>

How far each mode gets you:

- **Thin mode (default two pages)** — content-blind relays + broker, DHT discovery, no accounts. The code host is the client's TCB, pinned by SRI.
- **Owned mode (self-contained copy you run yourself)** — *plus* no code-host TCB: the code is yours, checkable against the published hash. The media path is unchanged.
- **Owned + your own relay (roadmap)** — *plus* no broker and no shared fleet: a fully party-less path where the only infrastructure you rely on is a relay you chose or run. Combined with self-signed tokens (which remove the broker's signing key and need no viewer token at all), this is the end state — self-hostable end to end.

The irreducible floor stays honest: **live video needs a relay** (real-time fan-out to many viewers can't be purely peer-to-peer at scale), and **any relay you connect to sees your IP**. The design's stance is to make every party either content-blind, swappable, or removable — never to pretend the relay isn't there.

*Code refs: client crypto — `src/crypto/{media-crypto,dht,node-id,link-keys}.ts` ; the two static pages `send.html` / `view.html` + the loaded client bundle; owned-copy build `vite.selfcontained.config.ts`

- `scripts/build-own.mjs` (§10); broker endpoints `/cdn/assign` and `/pub/salt` ; plus the relay installer script.*